

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_captioning_path = kagglehub.dataset_download('rehmang110/captioning')

print('Data source import complete.')
```

```
import torch
import numpy as np
import pandas as pd
import torchvision
from pathlib import Path
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image
import requests
import shutil
```

```
print(torch.__version__)
print(torchvision.__version__)
```

```
2.6.0+cu124
0.21.0+cu124
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16

else:
    device = "cpu"
    dtype = torch.float32

print(f"The device is :{device} , The Data type is : {dtype}")
```

```
The device is :cpu , The Data type is : torch.float32
```

```
# move the test dataset images from the input to working directory
src_path = Path("/kaggle/input/captioning")
dest_path = Path("/kaggle/working/test_images")
dest_path.mkdir(parents=True, exist_ok=True)

for file in src_path.iterdir():
    shutil.copy(file, dest_path / file.name)
```

```
# Load model and processor
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")
model.to(device)
```

[Show hidden output](#)

```
# load the image
# You can use a local image: Image.open("/path/to/your/image.jpg")
img_url = "https://storage.googleapis.com/sfr-vision-language-research/BLIP/demo.jpg"
image = Image.open(requests.get(img_url, stream=True).raw).convert('RGB')
```

```
image_path = Path("/kaggle/working/test_images/pexels-soc-nang-d-ng-2150345854-33267352.jpg")
image = Image.open(image_path).convert("RGB")
image
```



```
# Preprocess the image
inputs = processor(images=image, return_tensors="pt")

# Generate caption (greedy search)
out = model.generate(**inputs)
caption = processor.decode(out[0], skip_special_tokens=True)
print("📝 Caption:", caption)
```

📝 Caption: a pink flower with green leaves in the background